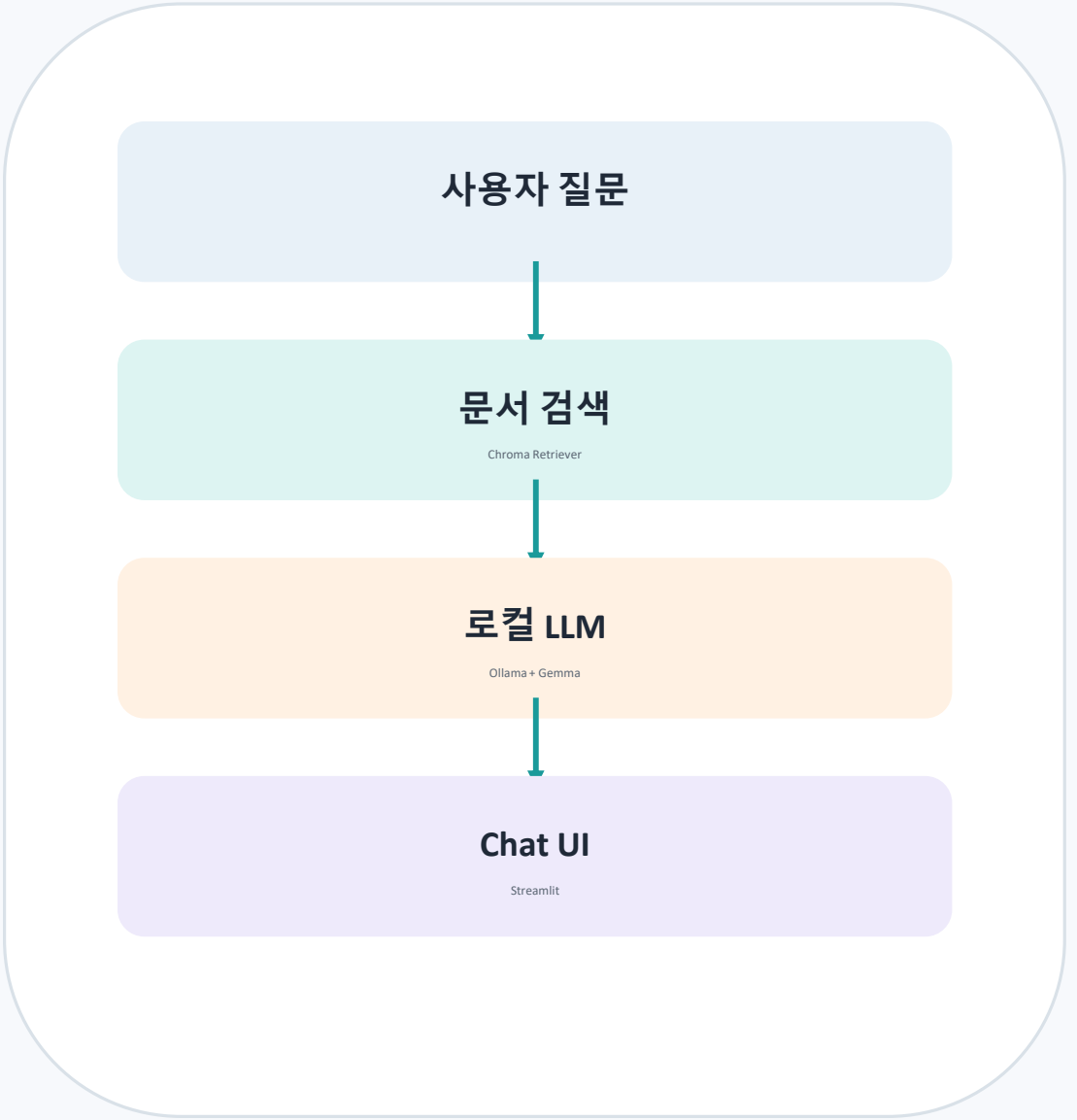
WEEK 7

RAG Chatbot

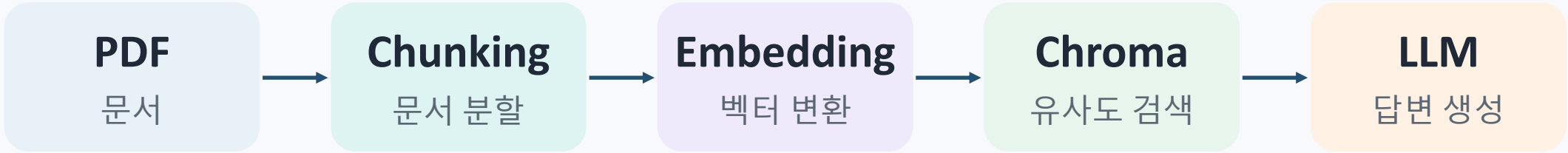
Basic RAG에서 대화형 RAG로



오픈소스 AI·SW 융합 프로젝트

6주차 Basic RAG 복습

문서를 검색하고, 검색된 내용을 근거로 한 번의 질문에 답변



질문 → 관련 문서 검색 → 검색된 문서 + 질문 → LLM → 답변

핵심: 외부 문서를 검색하여 답변의 근거로 활용

6주차 Basic RAG의 한계

한 번의 질문에는 답할 수 있지만, 연속된 대화의 맥락은 유지하지 않음

Basic RAG

사용자: 인턴십 관련 정보를 알려줘.

AI: 인턴십 기간은 11월 2일부터 12월 11일까지입니다.

사용자: 그럼 제출 기한은?

→ 현재 질문만 보면 “그럼”이 무엇을 가리키는지 알기 어려움

대화형 RAG에 필요한 기능

- 이전 질문과 답변 저장
- 현재 질문과 이전 질문 연결
- 대화 맥락을 반영한 문서 검색
- 연속 질문에 대한 답변 생성
- 채팅 형태의 사용자 인터페이스

Basic RAG → Conversational RAG

7주차 확장 목표

6주차의 문서 검색 구조를 유지하면서 대화 기능과 웹 UI를 추가

로컬 LLM 실행

Ollama + Gemma

대화 기록 유지

이전 질문·답변 저장

연속 질문 처리

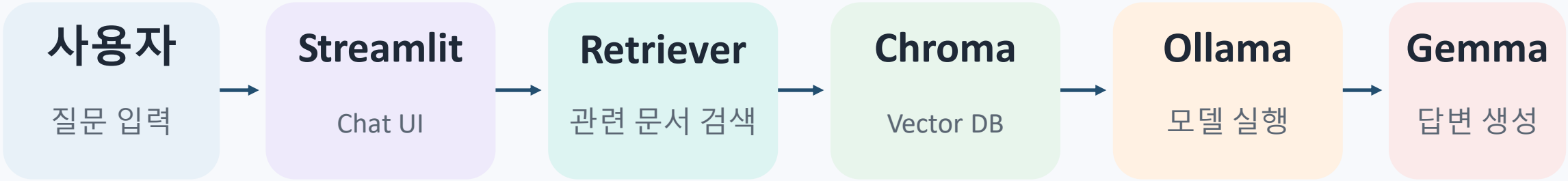
이전 질문 + 현재 질문

웹 Chat UI

Streamlit

문서 검색 + 대화 맥락 + 로컬 LLM + Chat UI

7주차 RAG Chatbot 전체 구조



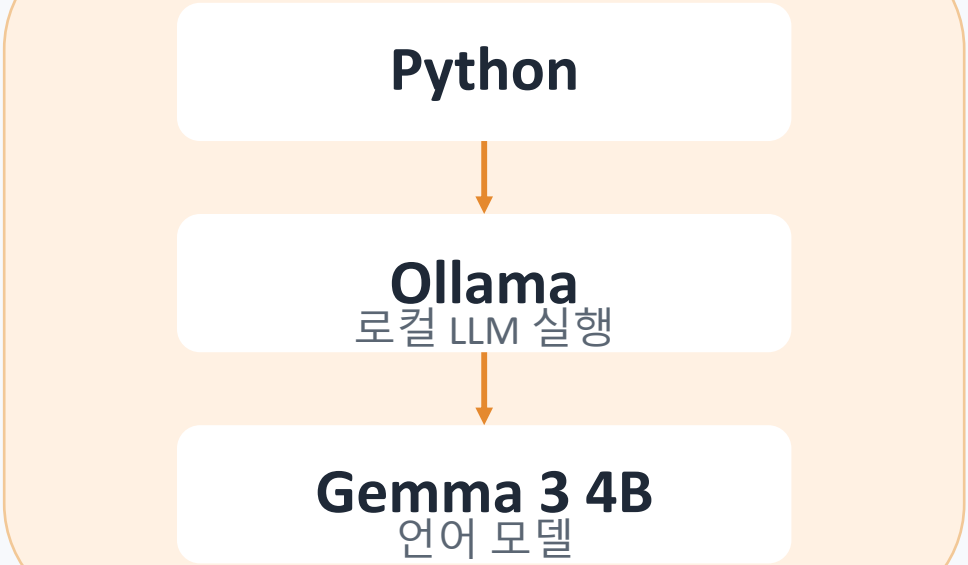
이전 대화 + 현재 질문 + 검색된 문서
→ Gemma가 문서 근거를 바탕으로 최종 답변 생성

Ollama

로컬 환경에서 LLM을 실행하고 프로그램과 연결하는 도구

주요 역할

- Gemma, Llama 등 LLM 다운로드
- 로컬 컴퓨터에서 모델 실행
- Python 프로그램에서 모델 호출
- 외부 API 없이 실습 가능



예: ollama pull gemma3:4b / ollama run gemma3:4b

Gemma

질문과 문서를 읽고 자연어 답변을 생성하는 언어 모델

Gemma3 4B

LLM
Large Language Model

텍스트 이해 · 생성

이번 실습에서 Gemma가 받는 정보

① 현재 질문

② 이전 대화 내용

③ Retriever가 찾은 관련 문서

이 정보를 함께 읽고 문서 기반 답변을 생성

Ollama와 Gemma의 관계

Ollama

모델을 실행하고 연결하는 도구

다운로드 · 실행 · Python 연결

실행

Gemma

실제로 언어를 처리하는 모델

질문 이해 · 답변 생성

Python 가상환경 (.venv)

프로젝트별로 Python 패키지와 버전을 분리하여 관리하는 실행 환경

컴퓨터의 Python 환경

프로젝트 A
.venv

2026-open / 7w
.venv

가상환경을 사용하는 이유

- 프로젝트에 필요한 패키지만 설치
- 다른 프로젝트와 패키지 충돌 방지
- 동일한 실습 환경 구성
- requirements.txt로 환경 재현

활성화되면 터미널 앞에 (.venv)가 표시됨

requirements.txt와 패키지 의존성

실습에 필요한 Python 패키지를 한 번에 설치하고 동일한 환경을 구성

```
streamlit==1.65.0

langchain-community
langchain-text-splitters
langchain-huggingface
langchain-chroma
langchain-ollama

chromadb
sentence-transformers
pypdf
torchvision
```

설치

python -m pip install -r requirements.txt

장점

패키지 누락을 줄이고, 학생마다 동일한 실습 환경을 구성할 수 있음

Streamlit

Python 코드로 웹 기반 Chat UI를 구현하는 프레임워크

```
st.title("RAG Chatbot")

question = st.chat_input(
    "질문을 입력하세요."
)

with
st.chat_message("assistant"):
    st.write(answer)
```

RAG Chatbot

사용자 질문

AI 답변

질문 입력 · 답변 출력 · 대화 기록 표시

Session State

Streamlit에서 이전 대화 내용을 유지하기 위한 상태 저장 공간

st.session_state.messages

사용자: 인턴십 정보를 알려줘.

AI: 인턴십 기간은 ...

사용자: 그럼 제출 기한은?

AI: 제출 기한은 ...

대화형 RAG에서의 역할

- 이전 질문과 답변 저장
- 현재 질문과 이전 질문 연결
- 화면이 다시 실행되어도 대화 표시 유지
- 후속 질문의 맥락 제공

torchvision

PyTorch 기반 이미지 처리 기능을 제공하는 확장 라이브러리

주요 기능

- 이미지 변환 및 전처리
- 이미지 데이터셋 처리
- 컴퓨터 비전 모델 지원

PyTorch 생태계에서 이미지 관련 기능을 담당

이번 실습 환경에서의 역할

sentence-transformers와 transformers를 사용하
용하는 과정에서 관련 모듈이 참조될 수 있어
실습 패키지에 포함

requirements.txt에 함께 작성하여 설치 환경을
통일

torchvision은 RAG 검색이나 답변 생성 기능을 담당하지 않음

실습 단계와 개념 연결

하나씩 기능을 추가하며 최종 RAG Chatbot을 완성

STEP 1	**Ollama 연결**	Python → Ollama → Gemma
STEP 2	**Retrieval**	PDF → Embedding → Chroma
STEP 3	**Basic RAG**	검색 문서 → Gemma → 답변
STEP 4	**Conversational RAG**	이전 대화 + 현재 질문
STEP 5	**Streamlit Chat UI**	브라우저 기반 RAG Chatbot

7주차 핵심 정리

Basic RAG

한 번의 질문에 대해 관련 문서를 검색하고 답변 생성

Conversational RAG

이전 대화와 현재 질문을 연결하여 연속 질문 처리

Ollama + Gemma

로컬 환경에서 LLM 실행 및 답변 생성

Streamlit

RAG 기능을 웹 기반 Chat UI로 제공

가상환경 + requirements

동일한 Python 실습 환경 구성

6W 문서 기반 질의응답 → 7W 대화형 RAG Chatbot